Ordered Heap Reconstruction

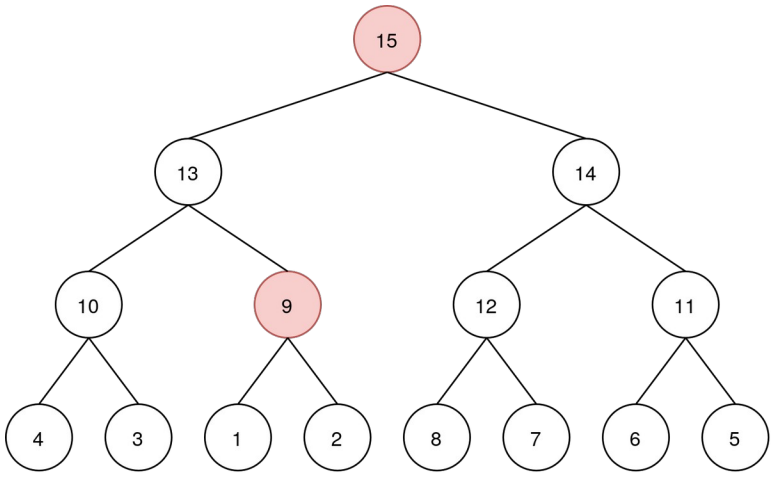
(1 sec, 512mb)

ข้อนี้จะนิยาม *Ordered Heap* ขึ้นมาใหม่ซึ่งจะมีข้อกำหนดเพิ่มเติมที่ต่างจาก Heap ที่เคยเรียนกันในห้องเรียนดังนี้

- mData[i] < mData[j] for all i < j สำหรับ min heap
- mData[i] > mData[j] for all i < j สำหรับ max heap

ให้นิสิตเขียนฟังก์ชัน std::vector<int> unordered_index() ให้กับคลาส CP::priority_queue ที่จะทำหน้าที่คืน index ของปมใน Heap ทั้งหมดที่ต้องสลับ subtree ซ้ายและ subtree ขวาของมันเพื่อให้ข้อมูลปัจจุบันใน Heap กลายเป็น Ordered Heap (index ที่คืนต้องเรียงจากน้อยไปมาก)

ตัวอย่าง



จากรูปภาพ max heap ด้านบน ฟังก์ชัน unordered_index() จะต้องคืนค่าออกมาเป็น {0, 4} นั่นคือจะต้องสลับ subtree ซ้าย-ขวา ของปมที่มีค่า 15 (index 0) และปมที่มีค่า 9 (index 4) เพื่อให้ข้อมูลดังกล่าวกลายเป็น ordered heap

การันตีว่า ข้อมูลเริ่มต้นใน heap จะสามารถเปลี่ยนให้เป็น ordered_heap ได้เพียงแค่การสลับ subtree ซ้าย-ขวา ของปมใดๆ และ ข้อมูลเริ่มต้นที่ให้จะเป็น full binary tree

ข้อบังคับ

- โจทย์ข้อนี้จะมีไฟล์ตั้งต้นมาให้ ซึ่งประกอบด้วยไฟล์ priority_queue.h, main.cpp และ student.h อยู่ ให้นิสิตเขียน code เพิ่มเติมลงในไฟล์ student.h เท่านั้น และการส่งไฟล์เข้าสู่ระบบ grader ให้ส่งเฉพาะไฟล์ student.h เท่านั้น
- ไฟล์ student.h จะต้องไม่ทำการอ่านเขียนข้อมูลใดๆ ไปยังหน้าจอหรือคีย์บอร์ดหรือไฟล์ใดๆ
- หากใช้ VS Code ให้ทำการ compile ที่ไฟล์ main.cpp
  **** main ที่ใช้จริงใน grader นั้นจะต่างจาก main ที่ได้รับในไฟล์โปรเจ็กต์เริ่มต้น แต่จะทำการทดสอบในลักษณะเดียวกัน ****

(มีต่อหน้าถัดไป)

คำอธิบายฟังก์ชัน main

main() จะอ่านข้อมูลนำเข้าดังนี้

- บรรทัดแรก รับจำนวนเต็ม N แทนจำนวนข้อมูลใน Heap
- N บรรทัดถัดมา แทนการ insert ข้อมูลเข้าไปใน Heap ตามลำดับ

หลังจากรับข้อมูลเสร็จ main() จะเรียกใช้ฟังก์ชัน unordered_index() และแสดงผลค่าที่ได้คืนมาจากฟังก์ชันดังกล่าว หากค่าที่คืนออกมาเป็นเวกเตอร์ว่าง โปรแกรมจะแสดงผลคำว่า “None”

ชุดข้อมูลทดสอบ

- 10% N = 3
- 20% N <= 255
- 30% ปมที่ต้องสลับจะมีอย่างมากปมเดียว
- 40% ไม่มีข้อกำหนดพิเศษอื่นใด

ตัวอย่างการทำงานของ main

ข้อมูลนำเข้า	ข้อมูลส่งออก
7 7 5 6 2 1 4 3	0
7 25 23 19 14 12 6 8	2
7 82 60 45 24 23 13 -2	None
15 15 13 14 10 9 12 11 4 3 1 2 8 7 6 5	0 4